

Risposte Discorsive – Sistemi Intelligenti

Teoria + Come rispondere all'orale

A.A. 2025–2026

Contents

1	Glossario degli Acronimi	2
2	Test di Turing e IA Forte vs Debole	4
3	Agenti e Ricerca	6
4	Algoritmi di Ricerca	7
5	CSP – Problemi di Soddisfacimento di Vincoli	12
6	Ricerca Avversariale	17
7	Logica del Primo Ordine (FOL)	19
8	Machine Learning	24
9	Matrice di Confusione	28
10	Ontologie	29
11	Frame Problem e Situation Calculus	30

1 Glossario degli Acronimi

Generali e Logica

Sigla	Significato	Spiegazione rapida
IA	Intelligenza Artificiale	Campo di studio: sistemi che esibiscono comportamento intelligente.
ML	Machine Learning	Apprendimento automatico: il sistema impara dai dati senza regole esplicite.
KB	Knowledge Base	Base di conoscenza: insieme di fatti e regole in forma logica.
FOL	First-Order Logic	Logica del Primo Ordine: logica con variabili, quantificatori ($\forall$, $\exists$) e funzioni.
CNF	Conjunctive Normal Form	Forma Normale Congiuntiva: congiunzione ($\wedge$) di clausole, ognuna disgiunzione ($\vee$) di letterali. Necessaria per la risoluzione.
CSP	Constraint Satisfaction Problem	Problema di Soddisfacimento di Vincoli: trovare un assegnamento completo e consistente per tutte le variabili.

Algoritmi di Ricerca

Sigla	Significato	Spiegazione rapida
BFS	Breadth-First Search	Ricerca in ampiezza: livello per livello. Completa e ottimale (costi uguali), spazio $O(b^d)$.
DFS	Depth-First Search	Ricerca in profondita': esplora un ramo fino in fondo. Spazio $O(bm)$, ma non completa ne' ottimale.
IDDFS	Iterative Deepening DFS	DFS con limite di profondita' crescente. Unisce la memoria di DFS ($O(bd)$) e la completezza/ottimalita' di BFS.
UCS	Uniform Cost Search	Ricerca a costo uniforme: frontiera ordinata per $g(n)$. Ottimale con costi variabili non negativi.
RBFS	Recursive Best-First Search	A* a memoria lineare $O(bd)$: puo' riesaminare nodi ma non mantiene tutta la frontiera in memoria.
A*	A-star	Ricerca con $f(n) = g(n) + h(n)$. Ottimale se h e' ammissibile (alberi) o consistente (grafi).

CSP e Inferenza Logica

Sigla	Significato		Spiegazione rapida
MRV	Minimum Values	Remaining	Euristica CSP fail-first: sceglie la variabile col dominio residuo piu' piccolo.
AC-3	Arc Consistency 3		Algoritmo per l'arc consistency: elimina valori senza supporto e propaga le riduzioni a cascata.
MPG	Modus Generalizzato	Ponens	Modus ponens per FOL: usa l'unificazione per applicare regole universali a fatti specifici.
MGU	Most General Unifier		Unificatore piu' generale: sostituzione θ che unifica due espressioni nel modo meno restrittivo possibile.
RDF	Resource Framework	Description	Standard Web Semantico: conoscenza come triple (soggetto, predicato, oggetto).

Machine Learning e Matrice di Confusione

Sigla	Significato		Spiegazione rapida
TP	True Positive		Vero positivo: istanza positiva classificata correttamente come positiva.
TN	True Negative		Vero negativo: istanza negativa classificata correttamente come negativa.
FP	False Positive		Falso positivo: istanza negativa classificata erroneamente come positiva.
FN	False Negative		Falso negativo: istanza positiva classificata erroneamente come negativa.
K-NN	K-Nearest Neighbours		Classificatore lazy: non costruisce un modello, usa i k esempi piu' vicini nel training set.
XOR	eXclusive OR		OR esclusivo: vero solo se i due input differiscono. Il perceptron non riesce ad impararlo perche' non e' linearmente separabile.
ID3	Iterative Dichotomiser 3		Algoritmo classico per alberi di decisione (variante dell'algoritmo di Hunt) basato sull'information gain.

2 Test di Turing e IA Forte vs Debole

Domanda: Cos'è il Test di Turing? Cosa dice esattamente?

Il Test di Turing, proposto da Alan Turing nel 1950 nel suo articolo “Computing Machinery and Intelligence”, è un criterio operativo per stabilire se una macchina può essere considerata intelligente. L'idea è la seguente: un giudice umano interagisce via tastiera con due interlocutori che non può vedere direttamente, uno dei quali è un essere umano e l'altro è una macchina. Se il giudice non riesce a distinguere l'uno dall'altro dopo una conversazione libera, allora la macchina supera il test e può essere considerata intelligente.

La cosa importante da sottolineare è che il test non richiede che la macchina risponda *correttamente* alle domande, né che le *comprenda* davvero: richiede che **imiti il comportamento umano** nel rispondere. Questo è un punto critico, perché un essere umano può sbagliare, può essere distratto, può rispondere in modo incoerente, e la macchina deve essere in grado di fare altrettanto in modo convincente.

Le capacità che un sistema dovrebbe avere per superare il test riguardano l'elaborazione del linguaggio naturale, la rappresentazione della conoscenza, il ragionamento automatico e l'apprendimento. John Searle ha criticato il test con l'argomento della Stanza Cinese: una persona che segue regole per manipolare simboli cinesi senza capirne il significato starebbe “superando” il test senza alcuna vera comprensione. Questo ha aperto il dibattito tra IA forte e IA debole.

Esempio

ELIZA (1966) era un programma che simulava uno psicoterapeuta riformulando le frasi dell'utente in domande: “Ho mal di testa” → “Perché dici di avere mal di testa?”. Molti utenti si convincevano di parlare con un essere umano – un precursore del Test di Turing. Non capiva nulla: seguiva solo pattern di testo.

Come ricordarlo

Turing = imita, non capisce. Il test misura il comportamento esterno, non l'intelligenza interna. Una fotocopia di un libro non “capisce” il libro – ma supera il test se la gente non sa distinguerla dall'originale.

Come dirlo all'orale

“Il Test di Turing, proposto nel 1950, è un criterio per stabilire se una macchina può essere considerata intelligente. Un giudice umano chatta con due interlocutori che non vede — uno umano, uno macchina — e se dopo la conversazione non li distingue, la macchina supera il test. Il punto fondamentale è che il test **non** richiede che la macchina capisca o risponda correttamente: richiede che *imiti* il comportamento umano in modo convincente. Un umano può sbagliare, essere distratto — e la macchina deve saper fare altrettanto.

Searle ha criticato questo con la Stanza Cinese: qualcuno che segue regole per manipolare simboli cinesi senza capirli starebbe superando il test senza alcuna comprensione reale. Questo apre il dibattito tra IA forte e debole.”

Domanda: Qual e' la differenza tra IA Forte e IA Debole?

Quando parliamo di IA debole intendiamo sistemi che *simulano* comportamenti intelligenti per uno scopo specifico, senza possedere comprensione reale o stati mentali propri. Un motore di scacchi, un sistema di riconoscimento vocale, un chatbot: questi sono tutti esempi di IA debole perche' risolvono problemi definiti, ma non “capiscono” quello che stanno facendo nel senso in cui lo fa un essere umano.

L'IA forte, al contrario, ipotizza che una macchina possa possedere vera intelligenza, coscienza e stati mentali equivalenti a quelli umani. Non si tratta di simulare, ma di *essere* intelligente. E' ancora un obiettivo teorico e filosofico, non raggiunto. Il libro di testo adotta l'approccio “**agire razionalmente**”: costruire agenti che massimizzano la misura di prestazione attesa, indipendentemente dal fatto che lo facciano “come un umano”.

Come ricordarlo

Debole = fa una cosa bene (ma non sa perche'). Deep Blue batte Kasparov a scacchi → IA debole: e' bravissimo a scacchi ma non sa allacciare le scarpe. **Forte** = capisce, ragiona su tutto come un umano → ancora fantascienza.

Come dirlo all'orale

“L'IA debole indica sistemi che simulano comportamenti intelligenti per uno scopo specifico senza comprensione reale: un motore di scacchi, un chatbot, un riconoscitore vocale. Risolvono problemi definiti, ma non capiscono quello che fanno nel senso in cui lo fa un umano.

L'IA forte ipotizza invece che una macchina possa avere vera intelligenza, coscienza e stati mentali equivalenti a quelli umani. Non si tratta di simulare, ma di *essere* intelligente. E' ancora un obiettivo teorico, non raggiunto.

Il libro di testo adotta un approccio pragmatico: costruire agenti che **agiscono razionalmente**, cioe' che massimizzano le prestazioni attese, indipendentemente dal fatto che lo facciano come un umano.”

3 Agenti e Ricerca

Domanda: Cos'è un agente razionale? Quali sono le componenti di un problema di ricerca?

Un agente è qualsiasi entità capace di percepire l'ambiente attraverso sensori e di agire su di esso tramite attuatori. Un agente è *razionale* quando sceglie l'azione che massimizza le sue prestazioni attese sulla base della sequenza di percezioni ricevute, della conoscenza disponibile e delle azioni possibili. È importante distinguere razionalità da onniscienza: un agente razionale fa del meglio che può con le informazioni che ha, ma non è infallibile perché il mondo contiene sempre fattori ignoti.

Quando formuliamo un problema di ricerca dobbiamo definire cinque elementi fondamentali. Lo **stato iniziale** è la configurazione di partenza dell'agente nel mondo. La **funzione successore** (o modello di transizione) descrive gli effetti di ogni azione: dato uno stato s e un'azione a , restituisce il nuovo stato risultante. Il **test obiettivo** determina se uno stato è una soluzione. Il **costo del cammino** è la somma dei costi di ogni passo compiuto. Infine, una **soluzione** è la sequenza di azioni che porta dallo stato iniziale a uno stato obiettivo.

Questi elementi non vanno confusi con un CSP: in un problema di ricerca cerchiamo una *sequenza di azioni*, mentre in un CSP cerchiamo un *assegnamento* che soddisfi dei vincoli, senza la nozione di cammino o costo.

Esempio

8-puzzle: stato iniziale = configurazione iniziale delle tessere; funzione successore = sposta la tessera vuota su/giù/sinistra/destra; test obiettivo = tessere in ordine 1-8; costo = numero di mosse. La soluzione è la *sequenza* di spostamenti, non un singolo valore.

Come ricordarlo

Ricerca = “come ci arrivo?” (sequenza di azioni). **CSP** = “cosa vale ogni variabile?” (assegnamento). Nel problema dell'8-puzzle cerchi le mosse; nel Sudoku cerchi i numeri nelle celle.

Come dirlo all'orale

“Un agente è qualsiasi entità che percepisce l'ambiente tramite sensori e agisce tramite attuatori. È *razionale* quando sceglie l'azione che massimizza le prestazioni attese con le informazioni disponibili. Razionalità non significa onniscienza: l'agente fa del meglio con quello che sa.

Per formulare un problema di ricerca servono cinque elementi: lo **stato iniziale**, la **funzione successore** che descrive gli effetti di ogni azione, il **test obiettivo** che dice se uno stato è soluzione, il **costo del cammino**, e la **soluzione** stessa, cioè la sequenza di azioni che porta all'obiettivo.

Attenzione a non confonderlo con un CSP: nella ricerca cerco una *sequenza di azioni*, nel CSP cerco un *assegnamento di valori* a variabili — senza cammino né costo.”

4 Algoritmi di Ricerca

Domanda: Descrivi BFS, DFS, IDDFS e UCS: completezza, ottimalità, complessità

Il **BFS** (Breadth-First Search) esplora il grafo per livelli: espande prima tutti i nodi a profondità d , poi quelli a profondità $d + 1$, e così via. Usa una coda FIFO come frontiera. E' **completo** – trova sempre una soluzione se esiste – e **ottimale** quando i costi degli archi sono tutti uguali, perché il primo cammino trovato è anche il più corto in numero di passi. Il difetto principale è la **complessità spaziale**: mantiene in memoria tutti i nodi del livello corrente, il che con fattore di ramificazione b e soluzione a profondità d richiede $O(b^d)$ nodi.

Il **DFS** (Depth-First Search) esplora in profondità lungo un ramo fino alla foglia, poi torna indietro e prova il successivo. Usa uno stack. Ha complessità spaziale $O(bm)$, dove m è la profondità massima – molto più efficiente del BFS. Tuttavia **non è completo** su grafi con cicli e **non è ottimale**: può trovare soluzioni molto profonde prima di scoprirne di più vicine.

L'**IDDFS** (Iterative Deepening DFS) combina i vantaggi di BFS e DFS: esegue ripetutamente una ricerca DFS limitata a profondità crescente $l = 0, 1, 2, \dots$ fino a trovare la soluzione. Ha la **completezza e l'ottimalità di BFS** (con costi uniformi) ma la **complessità spaziale di DFS**: $O(bd)$. Il costo di riesaminare i nodi già visitati è sorprendentemente basso – asintoticamente non cambia la complessità temporale $O(b^d)$.

L'**UCS** (Uniform Cost Search) è BFS con priorità: la frontiera è una coda a priorità ordinata per $g(n)$, il costo del cammino percorso. Estrae sempre il nodo di costo minimo. E' **completo e ottimale** anche con archi a costo variabile, purché i costi siano non negativi. Quando tutti i costi sono uguali a 1, degenera esattamente in BFS.

Esempio

Immagina un labirinto con corridoi di lunghezza diversa. **BFS** trova l'uscita con meno svolte; **DFS** imbocca il primo corridoio e va fino in fondo (può sbagliare tutto); **IDDFS** prova prima entro 1 corridoio, poi 2, poi 3... trovando la soluzione più breve; **UCS** tiene conto della lunghezza reale di ogni corridoio e trova il percorso più corto in metri.

Come ricordarlo

Memoria: BFS = Bulimica di RAM. DFS = Discende dritto ma si perde. IDDFS = il migliore dei due mondi. UCS = BFS con bilancia (pesa i costi). **DFS è l'unico non completo e non ottimale.**

Algoritmo	Completo	Ottimale	Tempo	Spazio
BFS	Si'	Si' (costi uguali)	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
IDDFS	Si'	Si' (costi uguali)	$O(b^d)$	$O(bd)$
UCS	Si'	Si'	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$
A*	Si'	Si' (euristica ammiss.)	$O(b^d)$	$O(b^d)$

Come dirlo all'orale

“Il **BFS** esplora per livelli con una coda FIFO. E' completo e ottimale con costi uguali, ma il difetto e' la memoria: mantiene tutti i nodi del livello corrente, $O(b^d)$ — impraticabile per spazi grandi.

Il **DFS** esplora in profondita' con uno stack. E' molto piu' efficiente in memoria, $O(bm)$, ma non e' completo su grafi con cicli e non e' ottimale: puo' trovare soluzioni profonde prima di quelle vicine.

L'**IDDFS** e' il migliore dei due mondi: esegue DFS con limite di profondita' crescente. Ha completezza e ottimalita' di BFS, ma memoria di DFS: $O(bd)$. Riesaminare i nodi sembra uno spreco ma asintoticamente non cambia la complessita' temporale.

L'**UCS** e' BFS con priorita': la frontiera e' ordinata per $g(n)$, il costo del cammino percorso. Completo e ottimale anche con costi variabili, purché non negativi. Con costi tutti uguali, degenera in BFS.”

Domanda: Cosa differenzia un algoritmo informato da uno non informato?

La differenza fondamentale e' l'utilizzo di una **funzione euristica** $h(n)$. Un algoritmo non informato come BFS, DFS o UCS non sa nulla sulla posizione del goal rispetto allo stato corrente: esplora il grafo in modo cieco. Un algoritmo informato, come A* o la ricerca greedy, dispone invece di una stima $h(n)$ del costo per raggiungere il goal da n e la usa per guidare l'esplorazione.

E' importante chiarire che la funzione costo $g(n)$ e' usata anche dagli algoritmi non informati come UCS. Quello che distingue gli algoritmi informati non e' il costo, ma proprio la stima euristica del costo futuro.

Come dirlo all'orale

“La differenza fondamentale e' l'uso di una **funzione euristica** $h(n)$. Un algoritmo non informato come BFS, DFS o UCS non sa nulla sulla posizione del goal: esplora alla cieca. Un algoritmo informato come A* dispone di una stima $h(n)$ del costo per raggiungere il goal e la usa per guidare l'esplorazione verso le zone piu' promettenti.

Vale la pena precisare: $g(n)$, il costo del cammino percorso, e' usato anche da UCS che e' non informato. Cio' che distingue gli algoritmi informati non e' il costo passato, ma la *stima del costo futuro*.”

Domanda: Cos'è un'euristica ammissibile? E una dominante?

Un'euristica $h(n)$ è **ammissibile** se non sovrastima mai il costo reale per raggiungere il goal: formalmente $h(n) \leq h^*(n)$ per ogni nodo n , dove $h^*(n)$ è il costo ottimo reale. La proprietà fondamentale è che A* con un'euristica ammissibile garantisce l'ottimalità su alberi di ricerca.

Se però lavoriamo su grafi con una lista chiusa abbiamo bisogno di una condizione più forte: la **consistenza** (o monotonicità). Un'euristica è consistente se $h(n) \leq c(n, n') + h(n')$ per ogni arco (n, n') . La consistenza implica l'ammissibilità, ma non viceversa.

Un'euristica h_1 **domina** h_2 se $h_1(n) \geq h_2(n)$ per ogni nodo n : h_1 è più informativa e A* con h_1 espande meno nodi.

Una funzione costante zero $h(n) = 0$ è tecnicamente ammissibile ma non informativa: A* degenera in UCS.

Esempio

Nell'**8-puzzle**: h_1 = numero di tessere fuori posto (ammissibile). h_2 = distanza di Manhattan (ammissibile e dominante su h_1 : stima più alta ma ancora ottimistica). A* con h_2 espande molti meno nodi.

Come ricordarlo

Ammissibile = **ottimista** = **non mente mai per eccesso**. Come un GPS che dice "arrivi in 10 minuti": può sbagliare per difetto ma non per eccesso. **Consistente**: $f(n)$ non scende mai lungo il cammino. **Domina**: $h_1 \geq h_2$ ovunque $\Rightarrow$ A* con h_1 è più furbo.

Come dirlo all'orale

"Un'euristica è **ammissibile** se non sovrastima mai il costo reale: $h(n) \leq h^*(n)$. La rende ottimistica — preferisce sempre soluzioni buone. A* con euristica ammissibile è ottimale su alberi.

Su grafi con lista chiusa serve qualcosa di più forte: la **consistenza**. Vale se $h(n) \leq c(n, n') + h(n')$ per ogni arco — la stima non può mai saltare più del costo reale di un passo. La consistenza implica l'ammissibilità, ma non viceversa. Con euristica consistente A* è ottimale anche su grafi.

h_1 **domina** h_2 se $h_1(n) \geq h_2(n)$ per ogni nodo: h_1 è più informativa, A* espande meno nodi. E $h = 0$ è ammissibile ma inutile — A* degenera in UCS."

Domanda: Come funziona A*? Perché è ottimale?

A* ordina la frontiera in base alla funzione $f(n) = g(n) + h(n)$, dove $g(n)$ è il costo del cammino già percorso da radice a n e $h(n)$ è la stima euristica del costo rimanente. A ogni passo espande il nodo con f minimo.

L'ottimalità di A* dipende dalla proprietà dell'euristica. Su **alberi di ricerca** basta l'ammissibilità. Su **grafi con lista chiusa** serve la **consistenza**, che garantisce

che f non diminuisca mai lungo un cammino: il primo percorso con cui si espande un nodo e' gia' quello ottimale.

La differenza tra A^* e UCS: UCS usa solo $g(n)$, A^* usa anche $h(n)$ per orientare la ricerca. Con $h = 0$, A^* degenera in UCS.

Formula di A^*

$f(n) = g(n) + h(n)$ dove $g(n)$ = costo percorso, $h(n)$ = stima euristica.
Ottimale se h ammissibile (alberi) oppure h consistente (grafi con lista chiusa).

Come dirlo all'orale

“ A^* ordina la frontiera per $f(n) = g(n) + h(n)$: costo percorso piu' stima del costo rimanente. Ad ogni passo espande il nodo con f minimo.
L'ottimalita' dipende dall'euristica: su alberi basta l'ammissibilita', su grafi con lista chiusa serve la consistenza, che garantisce che f non diminuisca mai lungo un cammino — quindi il primo percorso trovato per un nodo e' gia' ottimale.
La differenza con UCS e' esattamente questa: UCS usa solo $g(n)$ e non sa niente del goal. A^* usa anche $h(n)$ per orientarsi. Con $h = 0$, A^* degenera esattamente in UCS.”

Domanda: Cos'e' RBFS e quando si usa?

RBFS (Recursive Best-First Search) e' una versione di A^* progettata per operare con **memoria lineare**: mantiene in memoria solo il cammino corrente dalla radice al nodo attivo e i valori f dei nodi fratelli non ancora esplorati. Quando un ramo supera una soglia f_{limit} , l'algoritmo torna indietro, aggiorna il valore del nodo con il miglior f trovato, e poi ricomincia dal ramo con f minore.

Rispetto ad A^* , RBFS puo' riesaminare nodi piu' volte, aumentando il costo temporale; pero' la complessita' spaziale e' solo $O(bd)$. E' l'algoritmo da preferire quando la memoria disponibile e' il fattore limitante.

Come dirlo all'orale

“RBFS e' una versione di A^* a **memoria lineare**: tiene in memoria solo il cammino corrente e i valori f dei fratelli non esplorati. Quando un ramo supera la soglia f_{limit} , torna indietro, aggiorna il valore del nodo e riprende dal ramo con f minore.
Il compromesso e': puo' riesaminare nodi piu' volte — piu' lento in tempo — ma usa solo $O(bd)$ di spazio, come IDDFS. Si usa quando la memoria e' il fattore limitante.”

Domanda: Cos'e' la ricerca bidirezionale?

La ricerca bidirezionale esegue simultaneamente due ricerche in parallelo: una in avanti dallo stato iniziale e una all'indietro dallo stato obiettivo, fermandosi quando le due frontiere si incontrano. Invece di esplorare un albero di profondita' d con $O(b^d)$, si esplorano due alberi di profondita' $d/2$, riducendo la complessita' a $O(b^{d/2})$ — un risparmio esponenziale.

La difficoltà pratica è definire come effettuare la ricerca all'indietro: bisogna avere azioni invertibili e sapere in anticipo lo stato obiettivo.

Come dirlo all'orale

“La ricerca bidirezionale lancia due ricerche in parallelo: una in avanti dallo stato iniziale e una all'indietro dallo stato obiettivo, fermandosi quando le frontiere si incontrano. Invece di un albero di profondità d con $O(b^d)$, si esplorano due alberi di profondità $d/2$, con complessità $O(b^{d/2})$ — un risparmio esponenziale. La difficoltà pratica è la ricerca all'indietro: servono azioni invertibili e la conoscenza dello stato obiettivo.”

5 CSP – Problemi di Soddisfacimento di Vincoli

Domanda: Cos'è un CSP? Come si definisce una soluzione?

Un problema di soddisfacimento di vincoli (CSP) è una tripla $\langle X, D, C \rangle$ dove X è un insieme di variabili, D è l'insieme dei rispettivi domini e C è l'insieme dei vincoli. Ogni vincolo specifica quali combinazioni di valori sono ammissibili per un sottoinsieme di variabili.

Una soluzione di un CSP è un **assegnamento completo e consistente**: completo perché ogni variabile deve avere un valore, consistente perché nessun vincolo deve essere violato. Nel CSP non abbiamo né un cammino né un costo: vogliamo solo trovare una configurazione valida.

Esempio

Colorazione mappa Australia: variabili $X = \{WA, NT, SA, Q, NSW, V, T\}$, domini $D = \{Rosso, Verde, Blu\}$, vincoli: ogni coppia di regioni adiacenti deve avere colori diversi. Una soluzione: $WA = R, NT = G, SA = B, Q = R, NSW = G, V = R, T = G$.

Come ricordarlo

CSP = Sudoku: celle (variabili), valori 1–9 (domini), regole di riga/colonna/quadrato (vincoli). Non cerchi come arrivare alla soluzione – cerchi i valori giusti.

Come dirlo all'orale

“Un CSP è una tripla $\langle X, D, C \rangle$: variabili, domini e vincoli. Ogni vincolo specifica quali combinazioni di valori sono ammissibili.

Una soluzione è un assegnamento **completo** — ogni variabile ha un valore — e **consistente** — nessun vincolo violato. Non c'è né cammino né costo: vogliamo solo trovare una configurazione valida.

La differenza con la ricerca classica è netta: la' cerco una sequenza di azioni, qui cerco un assegnamento di valori.”

Domanda: Cos'è la commutatività nel CSP? Perché riduce l'ampiezza dell'albero?

La commutatività nei CSP afferma che l'**ordine in cui si assegnano le variabili non influenza la soluzione**. Questa proprietà è cruciale perché permette di costruire l'albero di ricerca assegnando una variabile alla volta invece di considerare tutte le possibili sequenze. Senza commutatività, l'albero avrebbe $n!$ percorsi distinti per ogni soluzione; con la commutatività se ne considera uno solo. Il fattore di ramificazione passa da $n \cdot d$ a d .

Come ricordarlo

Sudoku: assegnare prima A1 e poi B1 e' lo stesso che assegnare B1 e poi A1. L'ordine non cambia la soluzione finale – ma scegliere quale cella compilare prima (MRV) puo' risparmiarti un sacco di tentativi.

Come dirlo all'orale

“La commutativita' afferma che l'ordine in cui si assegnano le variabili non influenza la soluzione. E' cruciale perche' permette di costruire l'albero assegnando una variabile alla volta. Senza commutativita' ci sarebbero $n!$ percorsi per ogni soluzione; con essa se ne considera uno solo. Il fattore di ramificazione scende da $n \cdot d$ a d — una riduzione enorme.”

Domanda: Quali algoritmi si usano per risolvere un CSP? Qual e' il piu' e il meno efficiente?

L'approccio piu' basilare e meno efficiente e' il **generate and test**: si generano tutti i possibili assegnamenti completi e si verifica se soddisfano i vincoli. Complessita' $O(d^n)$.

Il **backtracking** migliora notevolmente: si assegnano le variabili una alla volta e, non appena un assegnamento parziale viola un vincolo, si torna indietro senza esplorare tutte le sue estensioni.

L'algoritmo piu' efficiente e' il **back-jumping**: invece di tornare sempre alla variabile precedente, identifica il **conflict set** – l'insieme delle variabili gia' assegnate che hanno causato il fallimento – e torna direttamente alla piu' recente tra queste.

Come dirlo all'orale

“L'approccio meno efficiente e' il **generate and test**: si generano tutti gli assegnamenti completi e si controlla se sono validi. Complessita' $O(d^n)$ — non sfrutta affatto i vincoli durante la costruzione.

Il **backtracking** assegna una variabile alla volta e torna indietro non appena un assegnamento parziale viola un vincolo. Molto meglio.

Il piu' efficiente e' il **back-jumping**: il backtracking cronologico torna sempre alla variabile precedente anche se non e' responsabile del conflitto. Il back-jumping identifica il *conflict set* — le variabili che hanno causato il fallimento — e salta direttamente alla piu' recente tra queste, evitando di esplorare rami inutili.”

Domanda: Cosa sono MRV e l'euristica di grado nel CSP?

La **MRV** (Minimum Remaining Values), detta fail-first, sceglie la variabile con il **dominio residuo piu' piccolo**: ha meno valori ammissibili rimanenti, quindi con piu' probabilita' portera' a un fallimento. Rilevare il fallimento presto significa non sprecare tempo su rami inutili.

L'**euristica di grado** serve come spareggio: si sceglie la variabile coinvolta nel maggior numero di vincoli con le variabili non ancora assegnate.

Esempio

Australia: SA tocca 5 altre regioni. MRV sceglie chi ha meno colori rimasti; euristica di grado dice: a parità, scegli SA perché ha più vincoli.

Come ricordarlo

MRV = fail-first: scegli chi rischia di bloccarti prima. **Euristica di grado = chi comanda:** scegli chi ha più vincoli con gli altri.

Come dirlo all'orale

“MRV e euristica di grado sono euristiche per scegliere l'ordine di assegnamento delle variabili, con l'obiettivo di rilevare i fallimenti il prima possibile.

La **MRV** sceglie la variabile con il dominio residuo più piccolo: è quella che con più probabilità porterà a un fallimento. Meglio scoprirlo subito che dopo aver assegnato altre variabili inutilmente.

L'**euristica di grado** serve come spareggio a parità di MRV: si sceglie la variabile con più vincoli verso le variabili non ancora assegnate. Ha l'impatto maggiore sulle scelte future — meglio vincolarla subito.”

Domanda: Cos'è il Forward Checking nel CSP?

Il forward checking opera *durante* la costruzione dell'assegnamento: ogni volta che si assegna un valore alla variabile X_i , si eliminano dai domini delle variabili future i valori incompatibili. Se un dominio diventa vuoto, si fa subito backtracking.

Tuttavia il forward checking non rileva le incompatibilità tra variabili future non ancora assegnate. Per questo esiste l'arc consistency.

Come dirlo all'orale

“Il forward checking è una tecnica di propagazione dei vincoli che opera *durante* la costruzione. Ogni volta che assegno un valore a X_i , elimino subito dai domini delle variabili future i valori incompatibili. Se un dominio si svuota, faccio backtracking senza dover assegnare altre variabili.

Il limite è che non rileva incompatibilità *tra* variabili future non ancora assegnate. Per quello serve l'arc consistency.”

Domanda: Cos'è l'Arc Consistency (AC-3)?

Un arco (X_i, X_j) è **arc-consistent** se per ogni valore nel dominio di X_i esiste almeno un valore nel dominio di X_j compatibile con il vincolo. L'algoritmo **AC-3** gestisce una coda di archi: prende (X_i, X_j) , rimuove da D_i i valori senza supporto in D_j . Se D_i si riduce, aggiunge alla coda tutti gli archi (X_k, X_i) , propagando a cascata. Termina quando la coda è vuota o un dominio è vuoto.

La differenza con il forward checking: AC-3 propaga le riduzioni tra *tutte* le coppie di variabili, non solo tra quella appena assegnata e le future.

Come dirlo all'orale

“Un arco (X_i, X_j) e' arc-consistent se per ogni valore in D_i esiste almeno un valore in D_j compatibile. AC-3 mantiene una coda di archi da verificare: per ogni arco (X_i, X_j) rimuove da D_i i valori senza supporto in D_j . Se D_i si riduce, aggiunge alla coda tutti gli archi (X_k, X_i) — perche' la riduzione potrebbe invalidare archi gia' controllati. Termina quando la coda e' vuota o un dominio e' vuoto.

La differenza con il forward checking e' che AC-3 **propaga a cascata** tra tutte le coppie, non solo dalla variabile appena assegnata. E' molto piu' potente, a costo di piu' computazione.”

Domanda: Cos'e' la Path Consistency?

La path consistency (o 3-consistenza) estende il concetto di arc consistency a **triple di variabili**: un insieme $\{X_i, X_j, X_k\}$ e' path-consistent se per ogni assegnamento consistente di X_i e X_j esiste un valore di X_k compatibile con entrambi.

La gerarchia e': node consistency (1 variabile) $\subset$ arc consistency (2 variabili) $\subset$ path consistency (3 variabili) $\subset \dots$ Piu' si sale, piu' si rilevano inconsistenze in anticipo ma a costo computazionale crescente.

Come dirlo all'orale

“La path consistency estende l'arc consistency a **triple** di variabili: $\{X_i, X_j, X_k\}$ e' path-consistent se per ogni assegnamento consistente di X_i e X_j esiste un valore di X_k compatibile con entrambi.

Esiste una gerarchia: node consistency (1 variabile), arc consistency (2 variabili), path consistency (3 variabili), e cosi' via. Piu' si sale, piu' inconsistenze si rilevano in anticipo, ma a costo computazionale sempre maggiore. Nella pratica, l'arc consistency e' spesso il miglior compromesso.”

Domanda: Cos'e' il back-jumping e il conflict set?

Quando una variabile X_k non puo' ricevere alcun valore senza violare un vincolo, il suo **conflict set** e': $CS(X_k) = \{X_j : j < k, \exists \text{ vincolo tra } X_j \text{ e } X_k\}$.

Il back-jumping salta direttamente all'ultima variabile nel conflict set, bypassando le variabili intermedie che non hanno contribuito al fallimento.

Esempio

Variabili $X_1, \dots, X_5$. X_5 fallisce perche' in conflitto solo con X_1 e X_3 . Il backtracking cronologico torna a X_4 (inutile). Il back-jumping salta direttamente a X_3 .

Come ricordarlo

Backtracking = torna dal vicino. **Back-jumping** = torna dal colpevole. Se litighi con qualcuno ma l'intermediario non c'entra, vai direttamente dal responsabile.

Come dirlo all'orale

“Quando X_k non può ricevere alcun valore, il suo **conflict set** è l'insieme delle variabili già assegnate in conflitto con lei. Il back-jumping salta direttamente all'ultima variabile nel conflict set, bypassando tutto quello che sta in mezzo e che non ha contribuito al fallimento.

È molto più efficiente del backtracking cronologico, che torna sempre alla variabile immediatamente precedente — anche se non è la responsabile.”

6 Ricerca Avversariale

Domanda: Come funziona l'algoritmo Minimax?

Minimax e' un algoritmo per giochi a somma zero a due giocatori con informazione perfetta. Un gioco a **somma zero** e' tale che il guadagno di un giocatore corrisponde esattamente alla perdita dell'altro.

Il funzionamento si basa sull'assunzione che entrambi i giocatori giochino in modo ottimale. L'algoritmo costruisce l'albero di gioco alternando turni di MAX e MIN. Alle foglie si applica la funzione di utilita' (stati terminali) o la funzione di valutazione EVAL (profondita' limite). I valori vengono poi **propagati dalle foglie verso la radice**: i nodi MAX scelgono il valore massimo tra i figli, i nodi MIN scelgono il minimo.

Minimax *non garantisce la vittoria*: suggerisce solo la prossima mossa ottimale assumendo che l'avversario risponda nel modo migliore.

Esempio

Albero a 2 livelli: MAX alla radice, MIN ai figli. MIN: $\min(3, 5) = 3$ e MIN: $\min(2, 9) = 2$. MAX sceglie $\max(3, 2) = 3$: va a sinistra.

Come ricordarlo

Le foglie risalgono: MIN taglia i rami alti, MAX taglia i rami bassi. MAX = tu che vuoi il voto piu' alto. MIN = il professore che vuole dartene il minimo.

Formula Minimax

$$\text{MiniMax}(s) = \begin{cases} \text{Utility}(s) & \text{se } s \text{ e' terminale} \\ \max_a \text{MiniMax}(\text{Result}(s, a)) & \text{se } \text{Player}(s) = \text{MAX} \\ \min_a \text{MiniMax}(\text{Result}(s, a)) & \text{se } \text{Player}(s) = \text{MIN} \end{cases}$$

Complessita' temporale: $O(b^m)$. Con alpha-beta pruning nel caso migliore: $O(b^{m/2})$.

Come dirlo all'orale

“Minimax e' per giochi a somma zero a due giocatori con informazione perfetta — scacchi, tris, dama. Somma zero significa che il guadagno di uno e' esattamente la perdita dell'altro.

Si assume che entrambi i giocatori giochino in modo ottimale. Si costruisce l'albero di gioco alternando MAX e MIN. Alle foglie si applica la funzione di utilita' o la funzione di valutazione EVAL per gli stati non terminali. Poi i valori **risalgono dalle foglie verso la radice**: MAX prende il massimo tra i figli, MIN prende il minimo.

Importante: Minimax non garantisce la vittoria. Suggerisce solo la *prossima*

mossa ottimale assumendo che l'avversario risponda nel modo migliore. Se la posizione di partenza e' gia' sfavorevole, non c'e' niente da fare."

Domanda: Cos'e' l'Alpha-Beta Pruning? Come migliora Minimax?

L'alpha-beta pruning elimina rami dell'albero che non possono influenzare la decisione finale, senza cambiare il risultato. Si mantengono due valori: α (il miglior valore garantito finora per MAX) e β (il miglior valore garantito finora per MIN).

La regola: se durante un nodo MIN troviamo un valore $\leq \alpha$, tagliamo (MAX ha gia' un'alternativa migliore). Se durante un nodo MAX troviamo un valore $\geq \beta$, tagliamo (MIN non permettera' mai a MAX di arrivare qui).

Nel caso migliore la complessita' scende da $O(b^m)$ a $O(b^{m/2})$: si puo' cercare al doppio della profondita'. Alpha-beta *non cambia la mossa scelta*: produce lo stesso risultato di Minimax.

Come ricordarlo

α = il meglio garantito a MAX (cresce). β = il meglio garantito a MIN (decresce).
Taglia quando $\alpha \geq \beta$. Hai gia' un'offerta a 10k€. Se un concessionario parte da 15k€ senza sconti, non tratti nemmeno.

Come dirlo all'orale

"L'alpha-beta pruning ottimizza Minimax eliminando rami che non possono influenzare la decisione finale — *senza cambiare il risultato*. Si tengono due valori: α , il miglior valore garantito a MAX finora, e β , il miglior garantito a MIN.

La regola di taglio: se durante un nodo MIN trovo un valore $\leq \alpha$, taglio — MAX ha gia' un'alternativa migliore altrove. Se durante un nodo MAX trovo un valore $\geq \beta$, taglio — MIN non permettera' mai a MAX di arrivare qui.

Nel caso migliore — nodi ordinati con le mosse migliori per prime — la complessita' scende da $O(b^m)$ a $O(b^{m/2})$: posso cercare al doppio della profondita' con lo stesso calcolo. Nel caso peggiore, nessun vantaggio.

Fondamentale: alpha-beta produce *esattamente la stessa mossa* di Minimax. I rami tagliati non avrebbero mai potuto essere scelti da un giocatore razionale."

7 Logica del Primo Ordine (FOL)

Domanda: Quali simboli della FOL richiedono interpretazione?

In FOL distinguiamo simboli logici (quantificatori, connettivi, uguaglianza) che hanno un significato fisso, e simboli **non logici** che richiedono un'interpretazione verso il dominio del discorso.

I simboli non logici sono tre: i **simboli di costante**, i **simboli di funzione** e i **simboli di predicato**. Le variabili non richiedono interpretazione perché sono legate dai quantificatori.

Come dirlo all'orale

“In FOL distinguiamo simboli logici — quantificatori, connettivi, uguaglianza — che hanno significato fisso, e simboli non logici che richiedono un'interpretazione, cioè una mappatura verso il dominio del discorso.

I simboli non logici che richiedono interpretazione sono tre: **costanti** (denotano un oggetto specifico), **funzioni** (denotano una funzione dal dominio a se stesso) e **predicati** (denotano una relazione). Le variabili no, perché il loro significato dipende dall'assegnamento corrente dato dai quantificatori.”

Domanda: Cos'è la Forma Normale Congiuntiva (CNF)?

La CNF è una **congiunzione di clausole**, dove ogni clausola è una disgiunzione di letterali. Per esempio: $(A \vee \neg B) \wedge (B \vee C \vee \neg A)$ è in CNF. È fondamentale perché è la forma richiesta dall'algoritmo di risoluzione. Qualsiasi formula si può convertire in CNF: si eliminano bicondizionali e implicazioni, si porta la negazione verso l'interno con De Morgan, poi si distribuisce la disgiunzione sulla congiunzione.

Come dirlo all'orale

“La CNF è una rappresentazione standard delle formule logiche come **congiunzione di clausole**, dove ogni clausola è una disgiunzione di letterali. Un letterale è un atomo o la sua negazione.

È fondamentale perché è la forma richiesta dall'algoritmo di risoluzione. Qualsiasi formula si può convertire in CNF: prima si eliminano bicondizionali e implicazioni, poi si porta la negazione verso l'interno con De Morgan, infine si distribuisce la disgiunzione sulla congiunzione.”

Domanda: Cosa sono le clausole di Horn e perché sono importanti?

Una clausola di Horn contiene **al più un letterale positivo**. Sono importanti perché: l'inferenza su clausole di Horn è decidibile in tempo polinomiale, e ammettono algoritmi efficienti come forward e backward chaining.

In forma implicativa: $P_1 \wedge \dots \wedge P_k \Rightarrow Q$ — le parti negative sono premesse, la positiva è la conclusione. È la struttura delle regole SE... ALLORA... di Prolog

e dei sistemi esperti.

Esempio

$\neg \text{Umano}(x) \vee \text{Mortale}(x)$ e' di Horn (un solo positivo: Mortale). Equivale a $\text{Umano}(x) \Rightarrow \text{Mortale}(x)$.

$\text{Umano}(x) \vee \text{Robot}(x) \vee \neg \text{Costruito}(x)$ *non* e' di Horn: due letterali positivi.

Come ricordarlo

Horn = al piu' 1 testa = al piu' 1 conclusione. Prolog lavora solo con clausole di Horn: ogni clausola ha esattamente una testa (o nessuna, per i fatti).

Come dirlo all'orale

“Una clausola di Horn e' una clausola con **al piu' un letterale positivo**. Sono importanti per due ragioni: prima, l'inferenza su di esse e' decidibile in *tempo polinomiale*, mentre sulla FOL generale non e' nemmeno decidibile. Seconda, ammettono algoritmi efficienti come forward e backward chaining.

In forma implicativa: $P_1 \wedge \dots \wedge P_k \Rightarrow Q$. Le parti negative sono le premesse, l'unica positiva e' la conclusione. E' esattamente la struttura delle regole SE...ALLORA... dei sistemi esperti e di Prolog.”

Domanda: Come funziona il Forward Chaining?

Il forward chaining e' guidato dai **dati**: parte dai fatti noti e applica ripetutamente le regole le cui premesse sono soddisfatte, aggiungendo nuove conclusioni. Continua fino al goal o al punto fisso. Richiede clausole di Horn e che tutti i fatti nelle premesse siano presenti in KB.

Il **Modus Ponens Generalizzato (MPG)** e' la versione per FOL: usa l'unificazione per trovare la sostituzione θ che rende le premesse identiche ai fatti noti.

MPG

$$\frac{p_1, \dots, p_k \quad (P_1 \wedge \dots \wedge P_k \Rightarrow Q)\theta}{Q\theta}$$

dove θ unifica P_i con p_i per ogni i .

Come dirlo all'orale

“Il forward chaining e' guidato dai **dati**: parte dai fatti noti e applica ripetutamente le regole le cui premesse sono gia' soddisfatte, aggiungendo nuove conclusioni. Continua fino al goal o al punto fisso — quando non si possono piu' derivare nuovi fatti.

Richiede clausole di Horn, e *tutti* i fatti nelle premesse di una regola devono essere presenti in KB. Se anche uno solo manca, la regola non scatta.

Il Modus Ponens Generalizzato e' la versione per FOL: usa l'unificazione per trovare la sostituzione θ che rende le premesse identiche ai fatti noti, poi applica θ alla conclusione.”

Domanda: Cos'è il Backward Chaining? Differenze con il Forward Chaining?

Il backward chaining è guidato dall'**obiettivo**: parte dalla query e cerca a ritroso le regole la cui conclusione corrisponde alla query, poi tenta di dimostrare le premesse ricorsivamente. È la base di Prolog.

Forward parte dai fatti e deriva tutto il deducibile — può produrre fatti inutili. Backward parte dal goal e determina solo i fatti necessari — più efficiente con query specifiche. Forward preferibile con pochi fatti e molte query; backward con molti fatti e query specifiche.

Esempio

KB: $\text{Umano}(\text{Socrate}), \text{Umano}(x) \Rightarrow \text{Mortale}(x)$.

Forward: parto da $\text{Umano}(\text{Socrate})$, applico la regola, derivo $\text{Mortale}(\text{Socrate})$.

Backward: voglio $\text{Mortale}(\text{Socrate})$. Cerco regole con testa $\text{Mortale}(x)$, trovo la regola, sostituisco $\{x/\text{Socrate}\}$, mi chiedo: $\text{Umano}(\text{Socrate})$? Sì, è in KB. Fine.

Come ricordarlo

Forward = **detective** che parte dagli indizi. **Backward** = **avvocato** che parte dalla tesi e cerca le prove.

Come dirlo all'orale

“Il backward chaining è guidato dall'**obiettivo**: parte dalla query che voglio dimostrare e cerca a ritroso le regole la cui conclusione corrisponde, tentando poi di dimostrare le premesse ricorsivamente. È la strategia di Prolog.

La differenza con il forward è la direzione: il forward parte dai fatti e deriva tutte le conseguenze possibili — può produrre molti fatti inutili. Il backward parte dal goal e determina solo i fatti necessari per dimostrarlo — molto più efficiente quando la query è specifica.

Forward è preferibile con pochi fatti e molte query; backward con molti fatti e query specifiche.”

Domanda: Cos'è l'unificazione? Cosa si intende per MGU?

L'unificazione trova una sostituzione θ tale che due espressioni diventino sintatticamente identiche dopo averla applicata. Il **MGU** (Most General Unifier) è la sostituzione più generale: non vincola le variabili più del necessario.

L'unificazione fallisce con l'**occur check**: non si può unificare x con $f(x)$ (struttura infinita). Prolog omette l'occur check per efficienza.

Esempio

$\text{UNIFY}(P(x, f(y)), P(a, f(b))) = \{x/a, y/b\}$

$\text{UNIFY}(P(x), P(f(x))) = \text{fallisce}$ (occur check)

Come ricordarlo

Unificazione = trovare i valori che rendono due espressioni identiche. Come $P(x, 5) = P(3, y) \Rightarrow x = 3, y = 5$. MGU = la soluzione piu' generale.

Come dirlo all'orale

“L'unificazione e' il processo di trovare una sostituzione θ tale che due espressioni diventino sintatticamente identiche dopo averla applicata. Per esempio, $P(x, f(y))$ e $P(a, f(b))$ si unificano con $\{x/a, y/b\}$.

Il MGU e' la sostituzione piu' generale tra tutte quelle che unificano due espressioni: non vincola le variabili piu' del necessario. Da lui si possono derivare tutte le altre.

L'unificazione fallisce con l'occur check: non si puo' unificare x con $f(x)$ perche' si creerebbe una struttura infinita. Prolog omette questo controllo per efficienza, a rischio di risultati errati in certi casi.”

Domanda: Cos'e' la risoluzione e come si usa per la refutazione?

La risoluzione: date due clausole con un letterale e il suo complementare, produce il risolvente eliminando quei letterali. Da $(A \vee C)$ e $(\neg C \vee B)$ si ottiene $(A \vee B)$. In FOL si usa prima l'unificazione. Non richiede clausole di Horn.

La **refutazione**: per dimostrare $KB \models \alpha$, si aggiunge $\neg\alpha$ a KB in CNF e si applicano le risoluzioni fino alla clausola vuota $\square$ (contraddizione).

Come ricordarlo

Refutazione = **dimostrazione per assurdo**. Supponiamo α falso $\rightarrow$ contraddizione $\rightarrow$ quindi α e' vero. La clausola vuota $\square$ e' la contraddizione.

Come dirlo all'orale

“La risoluzione e' una regola di inferenza: date due clausole con un letterale e il suo complementare, produce una nuova clausola eliminando quei letterali. Da $(A \vee C)$ e $(\neg C \vee B)$ si ottiene $(A \vee B)$. In FOL si usa prima l'unificazione per rendere complementari i letterali. A differenza del forward chaining, funziona su qualsiasi clausola, non solo di Horn.

La dimostrazione per **refutazione**: per dimostrare $KB \models \alpha$, aggiungo $\neg\alpha$ a KB convertita in CNF e applico risoluzioni ripetutamente. Se arrivo alla clausola vuota $\square$ — contraddizione — allora $KB \wedge \neg\alpha$ e' insoddisfacibile, quindi α e' conseguenza logica di KB. E' una dimostrazione per assurdo.”

Domanda: Cosa sono i termini ground e le funzioni di Skolem?

Un termine e' **ground** se non contiene variabili. Le **funzioni di Skolem** eliminano i quantificatori esistenziali durante la conversione in CNF. Se $\exists x P(x)$ e' senza quantificatori universali esterni, si sostituisce x con una costante di Skolem k . Se e' dentro un universale — $\forall y \exists x P(x, y)$ — si sostituisce x con una funzione di Skolem $f(y)$, perche' il valore di x dipende da y . La skolemizzazione preserva la

soddisfacibilita', non la validita'.

Esempio

$$\exists x \text{ Gatto}(x) \rightarrow \text{Gatto}(k). \quad \forall y \exists x \text{ Padre}(x, y) \rightarrow \forall y \text{ Padre}(f(y), y).$$

Come ricordarlo

Costante di Skolem: qualcuno esiste $\rightarrow$ dagli un nome. **Funzione di Skolem:** per ogni Y esiste qualcuno $\rightarrow$ dipende da $Y \rightarrow$ usa $f(Y)$. **Ground** = senza variabili.

Come dirlo all'orale

“Un termine e' **ground** se non contiene variabili: solo costanti o funzioni applicate a costanti. $\text{Padre}(\text{Giovanni})$ e' ground; $\text{Padre}(x)$ no.

Le **funzioni di Skolem** servono durante la conversione in CNF per eliminare i quantificatori esistenziali. Se ho $\exists x P(x)$ senza quantificatori universali esterni, sostituisco x con una costante di Skolem k — ‘qualcuno esiste, dagli un nome’. Se il quantificatore esistenziale e' dentro uno universale — $\forall y \exists x P(x, y)$ — sostituisco x con una funzione di Skolem $f(y)$, perche' il valore di x dipende da y .

La skolemizzazione preserva la soddisfacibilita' della formula, non la validita'.”

Domanda: Cosa sono tautologie e contraddizioni?

Una **tautologia** e' vera in tutti i modelli possibili: $A \vee \neg A$ e' sempre vera. Le tautologie sono importanti perche' $KB \models \alpha$ se e solo se $(KB \Rightarrow \alpha)$ e' una tautologia.

Una **contraddizione** e' falsa in tutti i modelli: $A \wedge \neg A$. E' alla base della refutazione: se $KB \wedge \neg \alpha$ e' una contraddizione, allora $KB \models \alpha$.

Come dirlo all'orale

“Una **tautologia** e' una formula vera in tutti i modelli possibili, indipendentemente dai valori di verita' delle variabili. L'esempio classico e' $A \vee \neg A$: qualunque valore abbia A , e' sempre vera. Sono importanti perche' $KB \models \alpha$ se e solo se $(KB \Rightarrow \alpha)$ e' una tautologia.

Una **contraddizione** e' l'opposto: falsa in tutti i modelli. $A \wedge \neg A$ e' sempre falsa. E' alla base della dimostrazione per refutazione: se $KB \wedge \neg \alpha$ e' una contraddizione, allora α e' conseguenza logica di KB.”

8 Machine Learning

Domanda: Come si costruisce un albero di decisione? Cos'è l'algoritmo di Hunt?

L'**algoritmo di Hunt** è il metodo ricorsivo classico da cui derivano ID3 e C4.5. Si parte dall'intero training set e si verifica la condizione di terminazione: se tutte le istanze sono della stessa classe, si crea una foglia. Se le istanze sono poche (pre-pruning), si crea una foglia con la classe maggioritaria. Altrimenti si sceglie l'attributo con il massimo guadagno di informazione, si partizionano le istanze e si procede ricorsivamente.

Il **numero di istanze sotto una soglia** è un valido criterio di terminazione (pre-pruning). L'overfitting *non* è rilevabile durante la costruzione.

Esempio

Dataset Tennis: al nodo radice entropia ≈ 0.94 . $\text{Gain}(\text{Outlook}) \approx 0.246$ – il più alto. Outlook diventa la radice.

Come ricordarlo

Costruzione = scegli l'attributo che pulisce di più (massimizza info gain).
Stop = nodo puro **OPPURE** poche istanze. Mai fermarsi “perché c'è overfitting”: non lo puoi sapere in quel momento.

Come dirlo all'orale

“L'algoritmo di Hunt è il metodo ricorsivo classico per costruire alberi di decisione, da cui derivano ID3 e C4.5. Si parte dall'intero training set e si verifica la condizione di terminazione: se tutte le istanze sono della stessa classe, si crea una foglia con quella classe. Se le istanze sono poche — sotto una soglia di pre-pruning — si crea una foglia con la classe maggioritaria. Altrimenti si sceglie l'attributo che massimizza il guadagno di informazione, si creano nodi figli per ogni valore dell'attributo, si partizionano le istanze e si procede ricorsivamente. Un punto importante: il *numero di istanze sotto una soglia* è un valido criterio di terminazione. L'overfitting invece non è rilevabile durante la costruzione — non può essere usato come criterio di stop.”

Domanda: Cosa misura l'entropia? E l'Information Gain?

L'**entropia** misura l'impurezza in un insieme: quanto sono mescolate le classi. Zero = nodo puro, massima = classi distribuite uniformemente.

L'**information gain** misura la riduzione di entropia prodotta da uno split. Si sceglie l'attributo con gain massimo.

Ne' l'entropia ne' il guadagno si calcolano dalla matrice di confusione: appartengono alla *costruzione* del modello.

Entropia e Information Gain

$$H(t) = - \sum_i p(i|t) \log_2 p(i|t) \quad \text{Gain}(A) = H(\text{padre}) - \sum_v \frac{|D_v|}{|D|} H(D_v)$$

Come ricordarlo

Entropia alta = disordine. Moneta equa: $H = 1$ bit. Moneta sempre testa: $H = 0$. **Information gain = quanto ordine guadagni splittando.**

Come dirlo all'orale

“L'**entropia** misura il grado di impurezza in un insieme di dati: quanto sono mescolate le classi in un nodo. Se tutte le istanze appartengono alla stessa classe, l'entropia e' zero — nodo puro. Se le classi sono distribuite uniformemente, l'entropia e' massima.

L'**information gain** misura la riduzione di entropia prodotta da uno split: si calcola l'entropia del padre, si sottrae la media pesata delle entropie dei figli, e si sceglie l'attributo con gain massimo.

Un'attenzione: ne' l'entropia ne' l'information gain si calcolano dalla matrice di confusione. Appartengono alla fase di costruzione del modello, non alla sua valutazione.”

Domanda: Cosa si intende per overfitting? Cosa sono pre-pruning e post-pruning?

L'**overfitting** si verifica quando un modello impara anche il rumore del training set. Alta accuratezza sul training, basse prestazioni sui nuovi dati. Il principio di Occam suggerisce il modello piu' semplice a parita' di errore.

Il **pre-pruning** interrompe la ramificazione durante la costruzione (soglia di istanze o gain basso). Rischio: underfitting.

Il **post-pruning** si applica dopo la costruzione completa: si sostituiscono i sottoalberi che generalizzano male con foglie. Piu' affidabile del pre-pruning.

Come ricordarlo

Overfitting = sa la lezione a memoria ma non capisce. Pre-pruning = smetti di studiare in anticipo. Post-pruning = studia tutto, poi dimentica i dettagli inutili.

Come dirlo all'orale

“L'**overfitting** si verifica quando il modello impara anche il rumore e le irregolarita' casuali del training set invece del pattern sottostante. Risultato: alta accuratezza sul training, basse prestazioni sui dati nuovi. Il principio di Occam suggerisce che, a parita' di errore, si preferisca il modello piu' semplice.

Il **pre-pruning** previene l'overfitting durante la costruzione: si interrompe la ramificazione quando le istanze scendono sotto una soglia o il gain e' troppo

basso. Il rischio e' l'underfitting: fermarsi troppo presto.

Il **post-pruning** si applica dopo aver costruito l'albero completo: si esaminano i sottoalberi e si sostituiscono quelli che generalizzano male — verificato su un validation set — con foglie etichettate con la classe maggioritaria. E' piu' affidabile del pre-pruning perche' si ha visione globale dell'albero.

Il pruning riguarda *esclusivamente* il machine learning, non i CSP ne' i giochi.”

Domanda: Cos'e' il Perceptron? Quale problema lo ha reso famoso?

Il percettrone (Rosenblatt, 1957) e' il modello piu' semplice di neurone artificiale: calcola $\mathbf{w} \cdot \mathbf{x} + b$ e applica una funzione di attivazione. Implementa un test lineare: separa le classi con un iperpiano.

Il problema famoso e' l'**XOR**: Minsky e Papert (1969) dimostrarono che XOR non e' linearmente separabile — nessun iperpiano separa i casi positivi dai negativi. Questo porto' al primo “inverno dell'IA”.

Regola delta del Perceptron

$$w_j(t+1) = w_j(t) + \eta \cdot (d - o) \cdot x_j$$

η = learning rate, d = output desiderato, o = output prodotto.

Come ricordarlo

Percettrone = righello. Dati separabili con una retta: impara. XOR (angoli opposti di un quadrato): nessuna retta funziona. Soluzione: piu' strati.

Come dirlo all'orale

“Il percettrone, proposto da Rosenblatt nel 1957, e' il modello piu' semplice di neurone artificiale. Riceve un input, calcola la combinazione lineare pesata e applica una funzione di attivazione. L'apprendimento avviene con la regola delta: i pesi si aggiornano proporzionalmente all'errore.

Implementa un test lineare: i pesi definiscono un iperpiano che separa le classi. Il problema che ne ha messo in luce i limiti e' lo **XOR**. Minsky e Papert dimostrarono nel 1969 che XOR non e' linearmente separabile: le quattro combinazioni di input sono agli angoli di un quadrato, con i positivi agli angoli opposti, e nessuna retta li separa.

Questo porto' al primo inverno dell'IA, fino allo sviluppo del backpropagation per reti multistrato negli anni '80.”

Domanda: Cos'e' K-NN? Differenza con gli algoritmi che inducono regole?

K-NN e' un classificatore **lazy**: non costruisce nessun modello durante il training, memorizza tutto il dataset. Per classificare una nuova istanza, trova i k esempi piu' vicini e assegna la classe maggioritaria.

La differenza con algoritmi come ID3: questi costruiscono un modello generalizzato durante il training (albero, regole, iperpiano). K-NN non generalizza nulla: il

“modello” e’ l’intero training set.

Le strategie **Specific-to-General** partono da ipotesi specifiche e le generalizzano. Le **General-to-Specific** fanno l’opposto. Entrambe inducono regole esplicite: K-NN no.

Come dirlo all’orale

“K-NN e’ un classificatore **lazy**: non costruisce nessun modello durante il training, si limita a memorizzare l’intero dataset. Per classificare una nuova istanza, calcola la distanza da tutti gli esempi, trova i k piu’ vicini e assegna la classe maggioritaria.

La differenza fondamentale con gli algoritmi che *inducono regole* — come ID3 o le strategie Specific-to-General e General-to-Specific — e’ che questi costruiscono un modello generalizzato durante il training: un albero, un insieme di regole, un iperpiano. Quello poi viene applicato alle istanze nuove. K-NN non generalizza nulla: il modello e’ l’intero training set.

Specific-to-General parte da ipotesi molto specifiche e le generalizza per coprire tutti gli esempi positivi. General-to-Specific fa l’opposto: parte da ipotesi generali e le specializza per escludere i negativi. Entrambe inducono regole esplicite — K-NN no.”

9 Matrice di Confusione

Domanda: Cos'è la matrice di confusione? Cosa si calcola da essa?

La matrice di confusione confronta etichette reali con quelle predette. Per classificazione binaria: **TP**, **FP**, **FN**, **TN**.

Prestazione ottimale = tutti i valori sulla diagonale principale (TP e TN). Le metriche: accuratezza = $(TP + TN)/\text{totale}$, error rate = $1 - \text{accuratezza}$, precisione e richiamo. Non si calcolano dalla matrice: entropia, information gain, overfitting.

Esempio

100 email: 90 spam, 10 normali. Classificatore “tutto spam”: TP=90, FP=10, FN=0, TN=0. Accuratezza = 90% ma TN=0: non riconosce mai un'email normale.

Come ricordarlo

Diagonale = indovinato. Fuori diagonale = sbagliato. FP = allarme falso. FN = mancato allarme. In medicina preferisci FP a FN.

Come dirlo all'orale

“La matrice di confusione confronta le etichette reali con quelle predette dal modello. Per classificazione binaria ho quattro celle: **TP** (predetto positivo, realmente positivo), **FP** (predetto positivo, realmente negativo), **FN** (predetto negativo, realmente positivo), **TN** (predetto negativo, realmente negativo). Una prestazione ottimale ha tutti i valori sulla diagonale principale — TP e TN — con zero fuori.

Le metriche calcolabili: accuratezza = $(TP + TN)/\text{totale}$, error rate, precisione e richiamo. Cio' che *non* si calcola dalla matrice e' l'entropia, l'information gain o l'overfitting: appartengono alla costruzione del modello, non alla valutazione.”

10 Ontologie

Domanda: Cosa si intende per ontologia? Quali relazioni contiene?

Un'ontologia è una specifica formale di una concettualizzazione. Le relazioni fondamentali sono quattro: **isa** (sottoclasse), **part-of** (composizione), **member** (istanza), e proprietà di classe ($\text{member}(X,C) \Rightarrow P(X)$).

Un insieme di sottocategorie disgiunte ed esaustive forma una **partizione**. RDF rappresenta la conoscenza come triple $\langle \text{soggetto}, \text{predicato}, \text{oggetto} \rangle$.

Esempio

Socrate **member** Filosofo. Filosofo **isa** Umano. Cervello **part-of** Umano. Per ereditarietà: Socrate è Umano $\Rightarrow$ Socrate è Mortale.

Come ricordarlo

isa = tipo di. member = individuo di. part-of = pezzo di. Errore tipico: confondere isa con member. Socrate non è una *categoria* di Filosofo — è un *individuo*.

Come dirlo all'orale

“Un'ontologia è una specifica formale di una concettualizzazione: definisce quali oggetti esistono in un dominio e come sono collegati.

Le relazioni fondamentali sono quattro. **isa** (is-a) esprime una sottoclasse: ‘virologo isa microbiologo’ significa che virologo è un tipo di microbiologo, e ogni virologo è anche un microbiologo. **part-of** esprime la composizione: ‘capsomero part-of virus’ indica che il capsomero è una componente strutturale. **member** esprime l'istanza: ‘HIV member virus’ indica che HIV è un esemplare specifico della classe virus, non una sottoclasse. Infine, ‘ $\text{member}(X,C) \Rightarrow P(X)$ ’ definisce una proprietà di classe.

Un insieme di sottocategorie disgiunte ed esaustive rispetto alla categoria padre forma una **partizione**. RDF, alla base del Web Semantico, rappresenta la conoscenza come triple soggetto-predicato-oggetto, non tramite clausole di Horn.”

11 Frame Problem e Situation Calculus

Domanda: Cos'è il Frame Problem?

Il frame problem è l'incapacità di sapere quali proprietà di una situazione si **preservano** dopo l'applicazione di un'azione. Non è descrivere cosa cambia — per quello bastano gli assiomi degli effetti — ma descrivere tutto ciò che *non* cambia senza elencare ogni singola proprietà invariata.

Nel **situation calculus** si risolve con gli **assiomi del successore di stato**: un fluente è vero nella situazione risultante se e solo se l'azione lo ha reso vero, oppure era già vero e l'azione non lo ha reso falso.

La definizione di un'azione include **precondizioni** ed **effetti**. Le proprietà atemporalì sono assiomi della KB generale.

Assioma del successore di stato

$$F(s') \Leftrightarrow [\text{l'azione ha reso } F \text{ vero}] \vee [F \text{ era vero in } s \wedge \text{l'azione non lo ha reso falso}]$$

dove $s' = \text{Do}(\text{azione}, s)$ è la situazione risultante.

Come ricordarlo

Frame problem = “**tutto il resto rimane uguale**” — **ma in logica devi dirlo!** Gli assiomi del successore di stato catturano sia gli effetti sia la persistenza in una sola formula.

Come dirlo all'orale

“Il frame problem è una delle sfide più sottili nella rappresentazione della conoscenza: è l'incapacità di sapere quali proprietà di una situazione si preservano dopo l'applicazione di un'azione. Il problema non è descrivere cosa cambia — per quello bastano gli assiomi degli effetti — ma descrivere tutto ciò che *non* cambia, senza dover elencare esplicitamente ogni singola proprietà invariata.

Per esempio: se un robot sposta un blocco, intuitivamente tutto il resto rimane uguale. Ma in logica formale, come so che il colore del muro non è cambiato? Senza un meccanismo apposito dovrei scrivere un assioma per ogni proprietà invariata.

Nel **situation calculus** questo si risolve con gli **assiomi del successore di stato**: un fluente è vero nella situazione risultante se e solo se l'azione lo ha reso vero, *oppure* era già vero e l'azione non lo ha reso falso. In una sola formula si catturano sia gli effetti sia la persistenza.

La definizione di un'azione include le precondizioni — ciò che deve valere per applicarla — e gli effetti — ciò che l'azione modifica. Le proprietà atemporalì non rientrano nella definizione dell'azione: sono assiomi generali della KB.”